



A STRUCTURED PLAYBOOK

# The path from single-digit to **100 PRs a day**

How a small game team can ship faster without sacrificing quality

**Jason Tu**

Thursday, September 24, 2026

YOU'VE SEEN THE CLAIMS

**"I'm shipping 100 PRs a day."**

But it isn't obvious how *your* team bridges the gap.

You don't work at a frontier lab

You don't have infinite tokens

You're not writing React

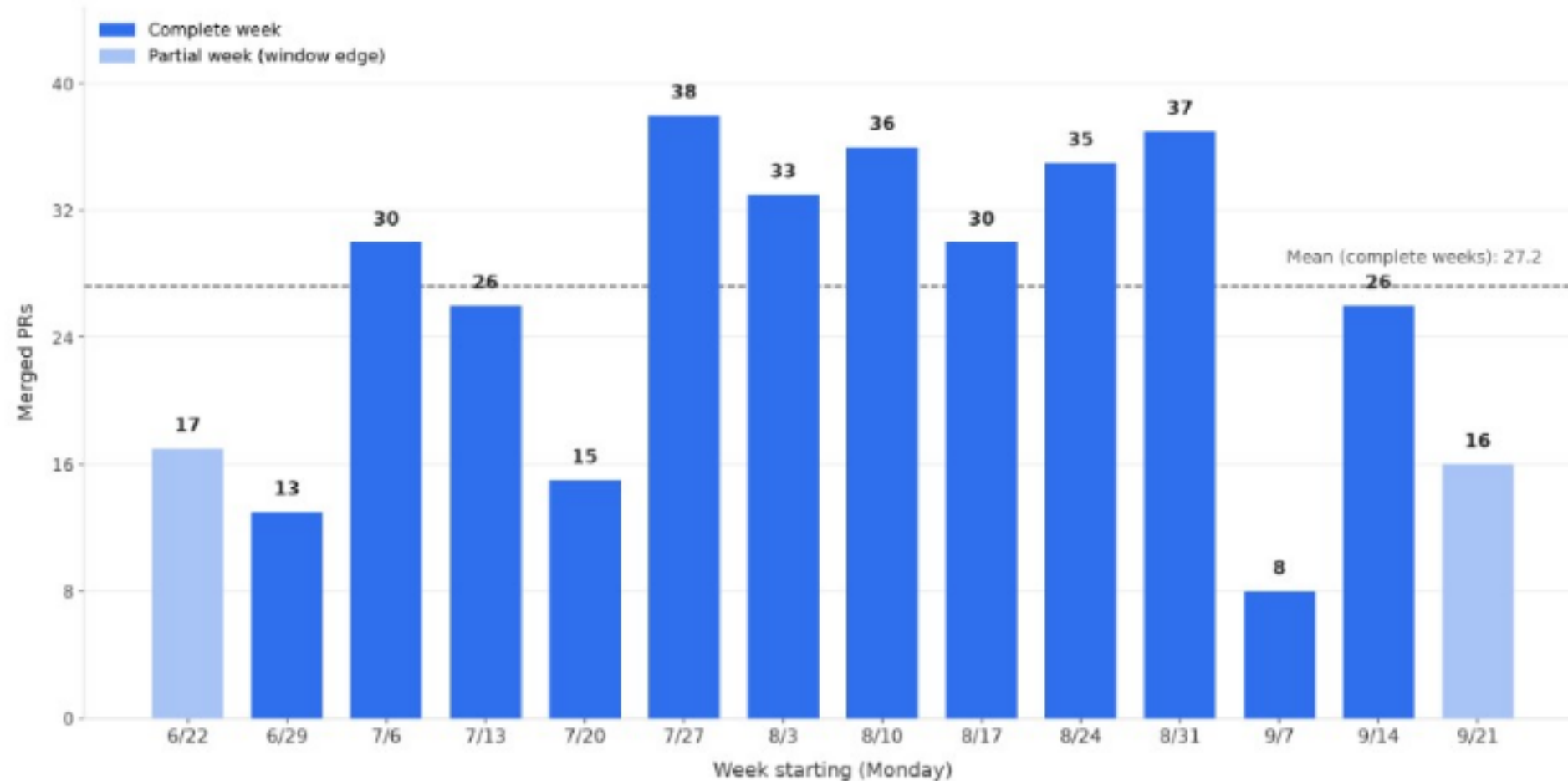


WHERE WE ARE TODAY

# Our PR throughput

## Merged PRs week over week — past 3 months

CardBattle - master - Jun 24 - Sep 24, 2026 - 360 total merges



SO YOU'RE SKEPTICAL

## Is this some kind of joke?

*"It's probably mostly trivial changes. I, too, can twiddle config files and open 10 PRs."*

*"Do we even have that many features to build?"*

*"PR count is a useless metric. What matters is getting the right things done."*

*"The code is probably bad. AI over-complicates things when a 1-line change would do."*



AND YET

## You'd love to use AI more, but...

- You know how to code, and you have opinions.
- You still find yourself **micromanaging**, pointing out things that don't match your high expectations.
- You might conclude AI just isn't as good as your artisanally crafted, human-written code.



A BETTER WORLD

# AI writes code while you sleep.

As outlandish as it sounds.

# See Card Battle *finally* reach global launch

- To get there **sooner** rather than in a few years.
- While maintaining our commitment to code and design **quality**.

But **how**? How do we have AI do our jobs well, that quickly, without churning out slop?

How much of the AI hype is BS, and how much isn't?



# This phenomenon is real.

01

## Understand

Map your pipeline: inputs, outputs, stages, baselines.

02

## Internalize

Develop an intuition for what agents actually are.

03

## Accelerate

Systematically speed up every stage of the pipeline.





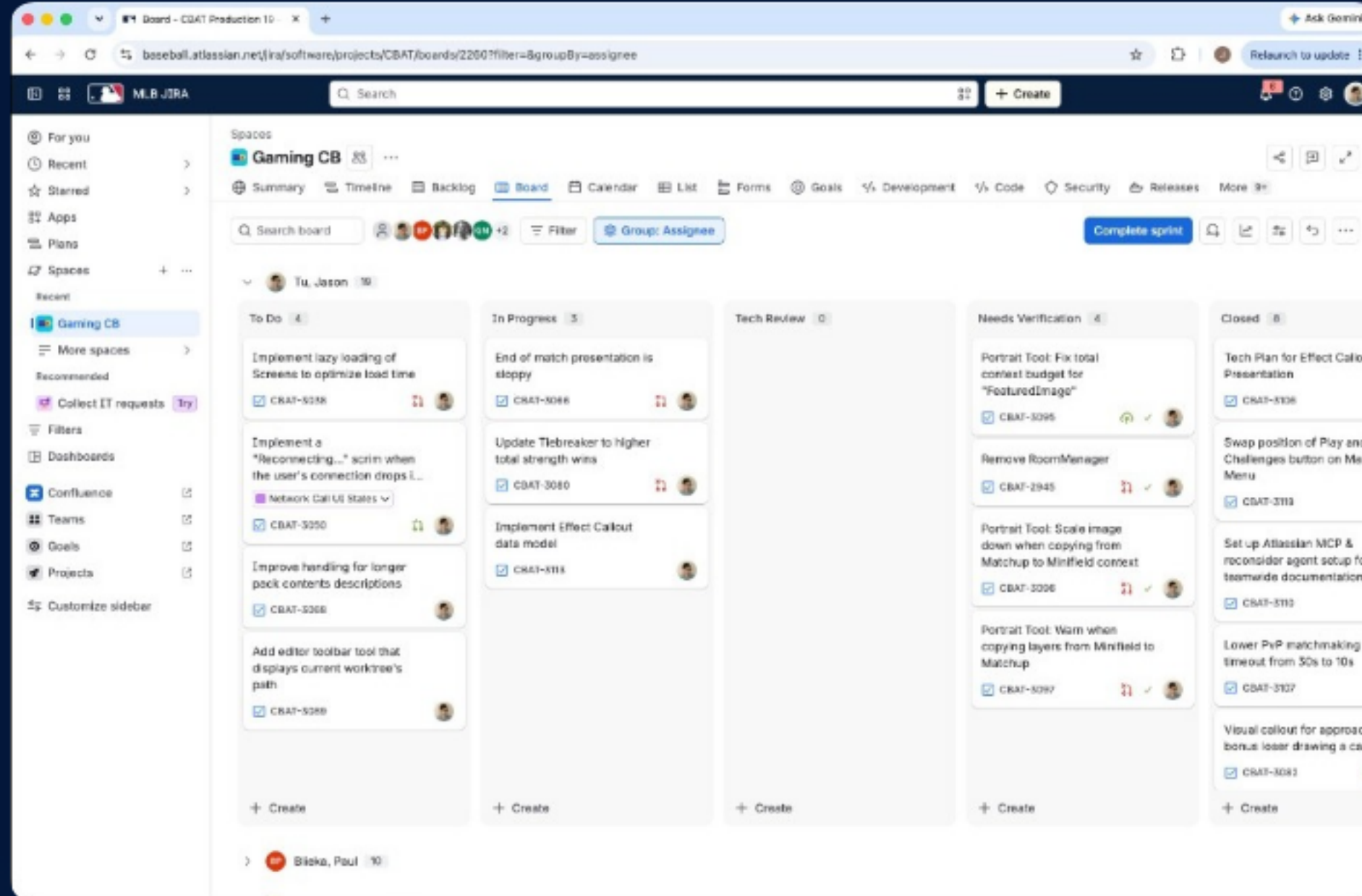
# 01

## Understand your pipeline

Define your inputs and outputs. Define the stages. Measure the throughput of each stage.

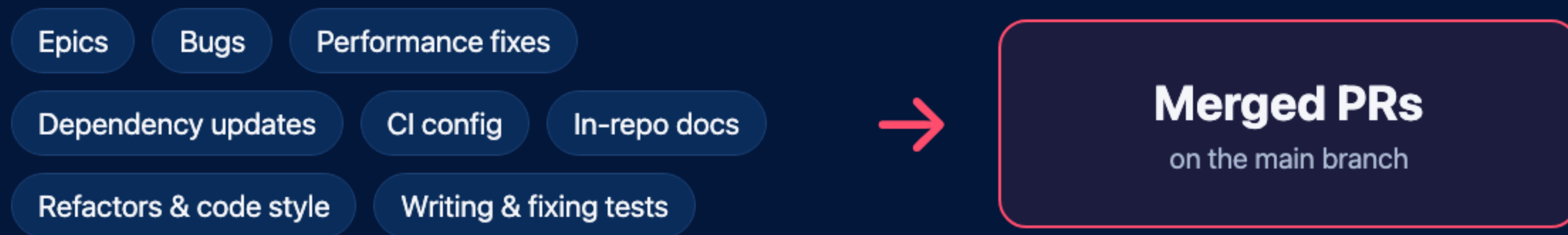
UNDERSTAND YOUR PIPELINE

# JIRA is our source of truth



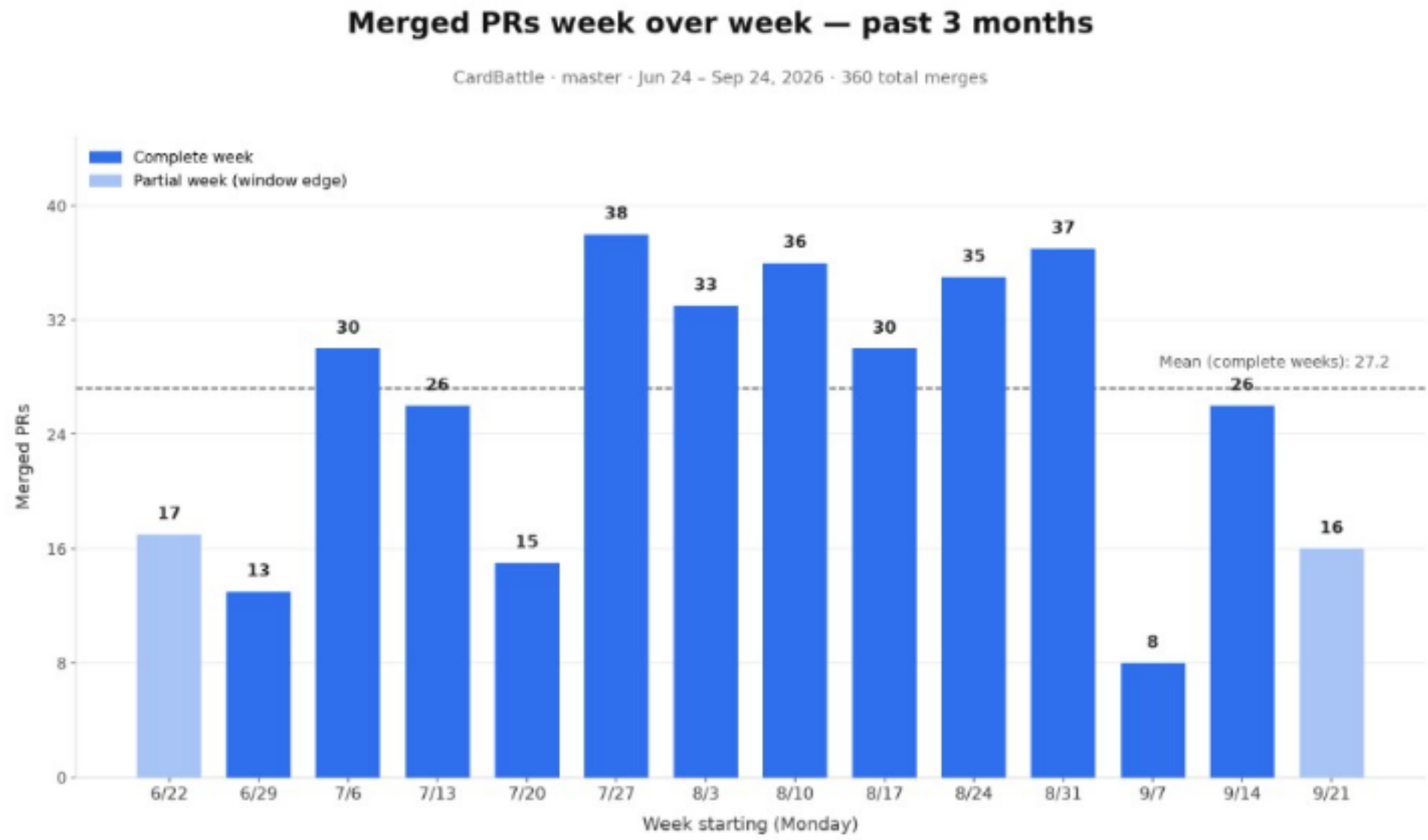
UNDERSTAND YOUR PIPELINE

# Inputs and outputs

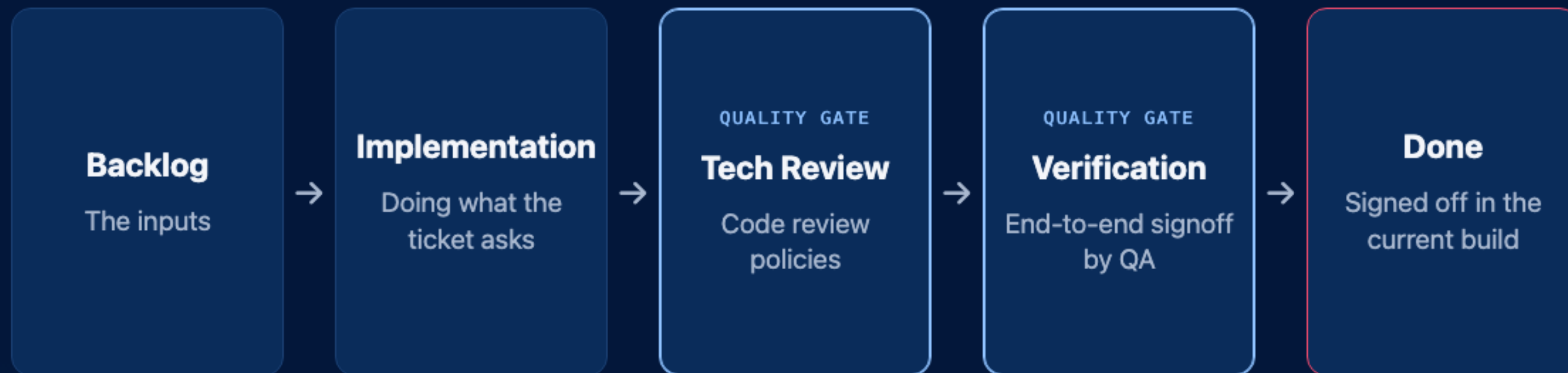


UNDERSTAND YOUR PIPELINE

# Establish a baseline



# The goal: maximize our team's throughput





02

# Develop an intuition for agents

To speed up the pipeline with AI, we need to understand what we're working with.

# You wield a **for** loop at your fingertips

```
class CodingAgent {
    async Task<ChatResponse> Run(string task) {
        Context context = await UnderstandCodebase();
        ChatResponse response = new ChatResponse();
        response.AddMessage("On it. Let me look into this...");

        while (!TaskComplete(task, context)) {
            Action action = await Think(task, context);
            Result result = await ExecuteTool(action);
            context.Add(result);
            response.AddAgentUpdate(result.Summary);
        }

        string finalAnswer = await FormatResponse(task,
context); response.AddMessage(finalAnswer);
        return response;
    }

    Result ExecuteTool(Action action) => action.Type switch {
        ToolType.ReadCode => ReadCode(action),
        ToolType.SearchCode => SearchCode(action),
        ToolType.EditFile => EditFile(action),
        ToolType.RunCommand => RunCommand(action),
        ToolType.Build => Build(action),
        ToolType.RunTests => RunTests(action),
        ToolType.InspectLogs => InspectLogs(action),
        ToolType.Git => Git(action),
    };
}
```



# Prompts set the loop's condition

Frame every prompt as a condition that is **currently unmet**, but **should be met** when the agent is done.



# Run many loops in parallel

Spin up worktrees. Name your sessions so you can keep track.



\* Muse and Grok Bot offer a glimpse of where agentic interfaces are headed, but we don't have access yet.



03

# Speed up every stage

Go through the pipeline systematically and ask: how can agents accelerate this?

## STAGE 1 • BACKLOG

# Curate your backlog

What lands here:

Epics

Bugs

Performance

Dependencies

CI config

Docs

Refactoring

- `atlassian-api` skill reads and updates JIRA tickets.
- `#gaming-cbat-qa` Slack messages automatically become bug tickets.
- A Confluence design doc gets broken down into prioritized tickets.
- Ticket descriptions get reformatted for coding agents.
- `/make-ticket` captures chores (deps, CI, docs, refactors).

The backlog is the input to your PR machine. Keep its quality high.



STAGE 2 · IMPLEMENTATION · SUBSTRATE

# AI-friendly architecture

Architecture that is hard to mess up, with clear interfaces, dataflow, and conventions.

## Model-View-Presenter

On the client

## Unified UI design system

One way to build UI

## Three-tier architecture

API, business logic, data layer

## Turn presentation timeline

Tweak turn phases independently

## Minifield sequences

Coordinators and applicators

## IMatchmaker

Keep local and remote implementations separate

Codify the principles in agent docs. Use **CODEOWNERS** as overseers of the architecture.



# Support architecture through skills

The key criterion: **one prompt to go from well-written JIRA ticket to opened PR.**

- Turn the prompts you keep typing over and over into reusable skills.
- If you're clicking around Unity manually, there is still more to automate.



## STAGE 2 · IMPLEMENTATION · SKILLS

# Card Battle skills, extracted from recurring prompts

## Backlog

`atlassian-api`

## Unity automation

`unity-cli`

## Git

`antislop``commit-and-push``draft-pr``gh-stack`

## UI creation

`create-ui-prefab``ui``ui-imgui``ui-ugui``ui-uitk``ui-verification``optimize-text-mesh-pro`

## Maintenance

`migrate-minifield-replay``migrate-screen-to-routable-screen``unity-package-management`

# Learn to identify loops in your work

## 1 · What's the condition?

What is the "done" condition? Be exact.

## 2 · How does the agent check it?

How can the agent check its own work without your help?

Give it the right tools (e.g. the Unity CLI) and a way to open a PR when done, and it runs **autonomously**. End every prompt with `/draft-pr`.



## STAGE 2 · IMPLEMENTATION · LOOPS

# Loops in practice

## iOS layout

Screenshot, change,  
screenshot again, compare.

2 weeks → 1–2 days

## Ability graphs

Generate graphs until they  
match the spec.

1 month → 2–3 days

## TDD

Write the failing test from my  
spec, then make it pass.

## Debug mode

Instrument with logs, invoke the  
code path, repeat until the issue  
is found.

## Framerate / CPU / memory

Profile for a baseline, change, re-profile. Repeat until performance  
requirements are met.



## EXAMPLE • IOS LAYOUT

# A prompt with a checkable condition

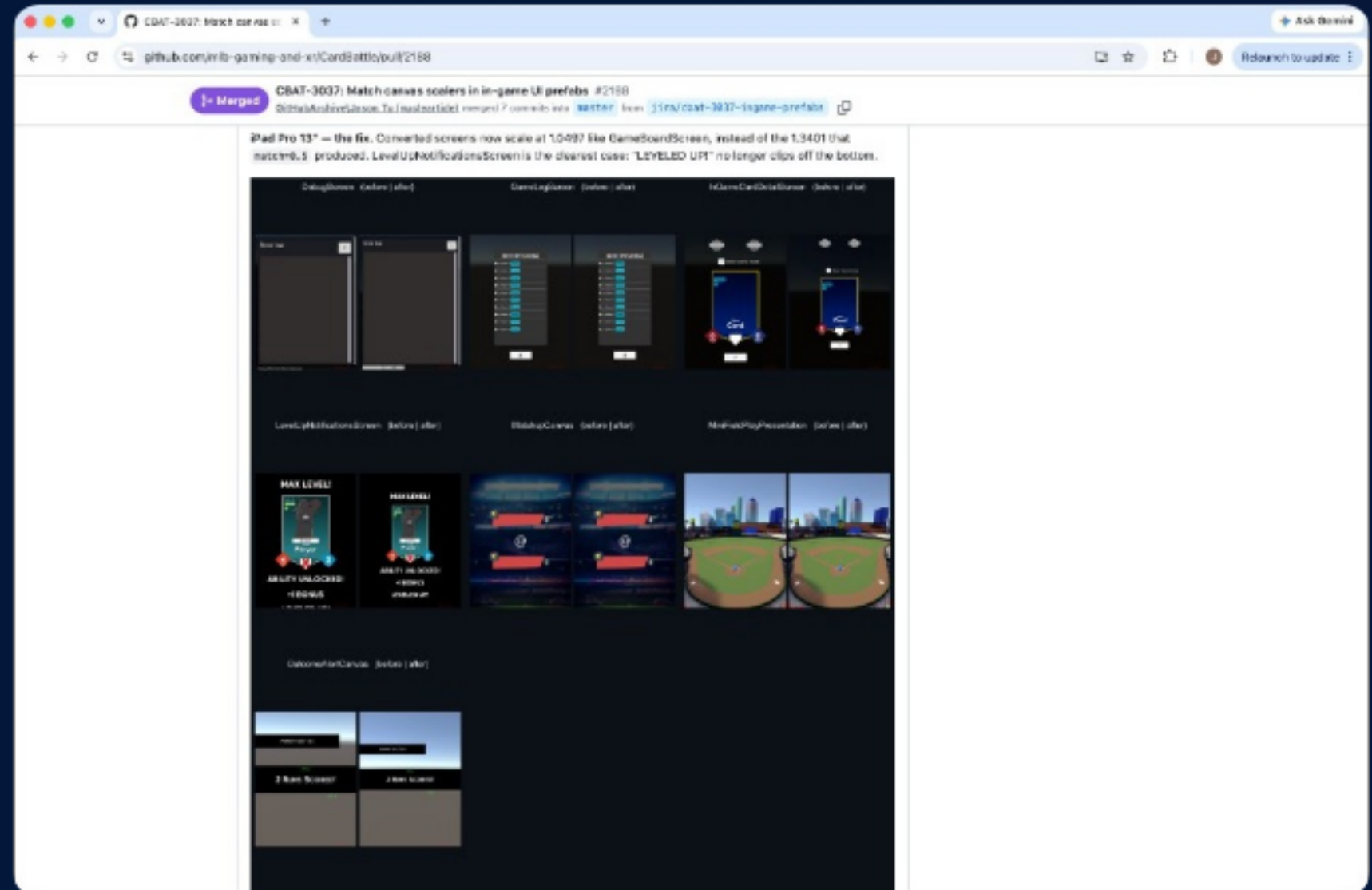
```
> prompt
```

Take screenshots of all the prefabs at iPhone resolution.

Make every Canvas Scaler consistent: Match Height = 1, reference resolution 1779 × 2556.

Take the screenshots again. They should be equivalent, but the Canvas Scaler settings should now be consistent across the board.

`/draft-pr` when you're done.



EXAMPLE · BUGS

# Bugs become loops, too

```
> prompt
```

```
[pasted screenshot]
```

```
Try to repro by writing a failing test in the right code path. Once  
you've got a failing test, make it pass.
```

```
/draft-pr when you're done.
```

- With Slack / Copilot automation, this runs the moment someone reports an issue.
- Juggle many bugs at once with worktrees. But watch your token usage.



## STAGE 3 • TECH REVIEW

# Human review is the biggest bottleneck

People are busy. PRs wait.

Major system decisions should **always** get human review, so the system doesn't go off the rails.

Everything else should be a skim if the PR:

**Doesn't invent architecture**

That's your job to design and oversee.

**Follows existing patterns**

Fits the architecture already in place.

**Passes automated tests**

Verified without a human in the loop.

**Passes static analysis**

Linters, compilers, Roslyn analyzers, formatters in CI.



## STAGE 3 • TECH REVIEW

# Compound your review comments



- Over time, you have fewer and fewer things to say.
- Review becomes a skim, because you've built trust in the system.
- Backlog → implementation → reviewed PR, **with no manual intervention.**



## STAGE 4 · VERIFICATION

# We are all SDETs now

Unity CLI can send taps to the editor and enter/exit Play Mode, so even end-to-end QA can be automated.

- Tag UI click targets with identifiers.
- Write `<X>FeatureMap.md` next to prefabs: how to drive them in a standalone Play Mode scene.
- Pause time and tick it forward in controlled increments.

I'd encourage you to write test instructions for your agent, and commit that to your PRs.



04

# Implications

# AI isn't perfect

## Output varies

Results differ between models.

## Micromanage first

Read the traces. Tweak docs and prompts until the agent meets your expectations.

## Tokens are finite

Limits dampen throughput, but don't change any of these techniques.



WHAT ARE WE IN THE END?

# We're in the managing business

Card Battle is a years-of-effort scope: multiplayer, PvP, mobile, with recurring content.

We're **6 engineers**. Each of us needs to punch far above our weight.

We design the architecture and process; our line chefs carry it out.



# Let the gains compound



PRs per day. Most days, we're below 10 today.

KNOW WHEN TO STOP

# Then focus on what actually matters

Past a point of diminishing returns, stop building the factory and spend the time on the game.

Game balance

The hero's journey & the baseball narrative

Playtesting

Fun

Metagame strategy

Synergy

Money



## TAKEAWAYS

# Treat the JIRA board as your pipeline, then speed up every stage

- **Backlog:** manage it with agents.
- **Substrate:** lay down architecture for agents to follow.
- **Implementation:** prompt with automated success-criteria loops.
- **Review:** feed human comments into compounding agent docs.
- **Verification:** automate it with agent-run end-to-end tests.



# Questions?